Oliver Vyyryläinen, Kevin Kovero, Edvárd Nagy

Fiksu Kukka

Metropolia University of Applied Sciences

Bachelor of Engineering

Electronics

Project Report

17 December 2025

Table of Contents

1. Introduction 1
2. System Architecture 2
   2.1 Master-Slave relationship and sleep mode 2
   2.2 Sensors 3
3. Design Development 5
   3.1 Initial Design 5
   3.2 Final Design 5
4. Results 5
5. Components used 6
6. Programming Codes for Arduino and ESP 7

1. Introduction

In today's fast-paced world, technology is increasingly being integrated into everyday life, even in areas we may not immediately consider, such as plant care. Our project "Fiksu kukka" is a smart plant system, designed to monitor and support the health of a plant. This approach combines sensors, automated systems, and data analysis to ensure that the plant receives the right amount of water and nutrients.

The project's goal was to implement a solution that included a sensor node, an edge device, an actuator and a human interface. The sensor node collects and processes data, sends it to the edge device and waits for instructions. The edge device stores the data, displays it to the user when requested and automatically sends commands to the sensor node to control the actuators if current moisture falls beneath a preset moisture level. The project was powered by an internal battery and housed in a 3D-printed plant pot.

2. Design Development

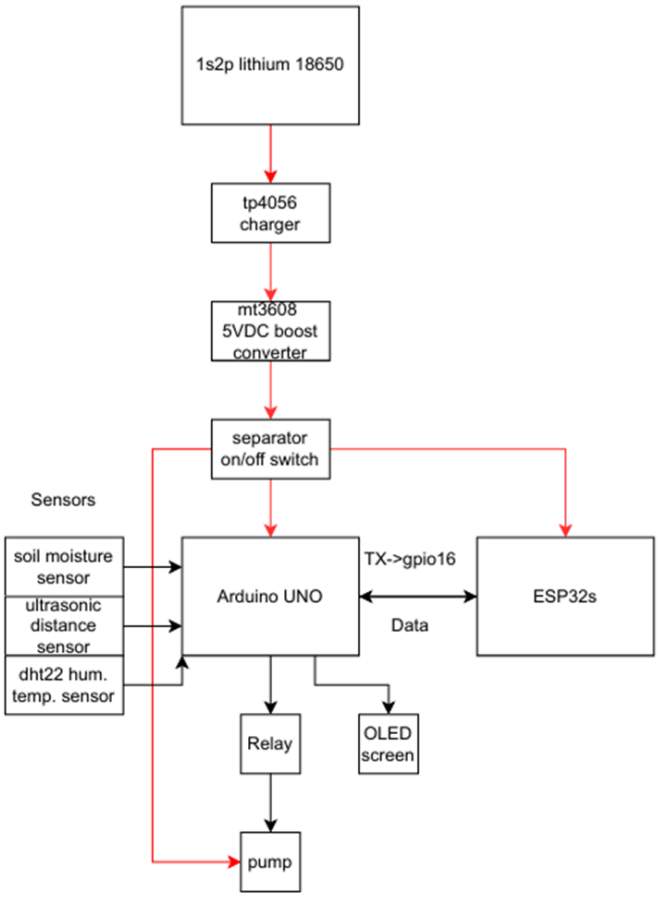
2.1 Initial Design

As a requirement for our project, we had to have a master and a slave, as well as multiple sensors to collect data with. Our first idea was to create a smart plant system that would collect data from sensors that would measure light level, soil moisture, temperature, humidity, as well as the water level of a cup, that would water the plant when necessary.

The only function the ESP had was to tell the Arduino to start collecting data, and the Arduino would then perform all the functionality of turning the water pump on if the water moisture level was too low. We also had no display to show sensor values.

2.2 Final Design

The design we settled on did not include the light level sensor, because we decided it was too unreliable. We also decided to have the Arduino send the collected data to the ESP, which in turn would then have an OLED display to show the sensor values.



Initial design diagram

3. System Architecture

3.1 Master-Slave relationship and sleep mode

The UNO was used as a sensor node in a master-slave relationship to an ESP32. The UNO is put in sleep mode and woken only to collect data and wait for instructions to actuate a relay. The UNO was set to wake from sleep once detecting a rising edge.

```
EICRA |= (1 << ISC01) | (1 << ISC00);
```

The bit combination 1 1 sets the EICRA to detect a rising edge. The External Interrupt Flag Register was used.

```
EIFR  |= (1 << INTF0);
```

Letting the register know that the flag is clear helps entering sleep and immediately waking up again. The INT0 pin was added to the External Interrupt Mask Register

```
EIMSK |= (1 << INT0);
```

Which allows the pin to wake the CPU from sleep mode. These are described in the ATmega328P datasheet under External interrupts.
Next the sleep mode is to be selected.

```
SMCR &= ~((1 << SM0) | (1 << SM2));
```

This clears any previous sleep mode. We select

```
SMCR |= (1 << SM1);
```

to choose the powerdown mode and

```
SMCR |= (1 << SE);
```

to enable the sleep mode. Then we re-enable interrupts and execute the sleep command.

The communication between the MCUs was done using a JSON string over UART by connecting the RX and TX pins. The JSON string is formatted

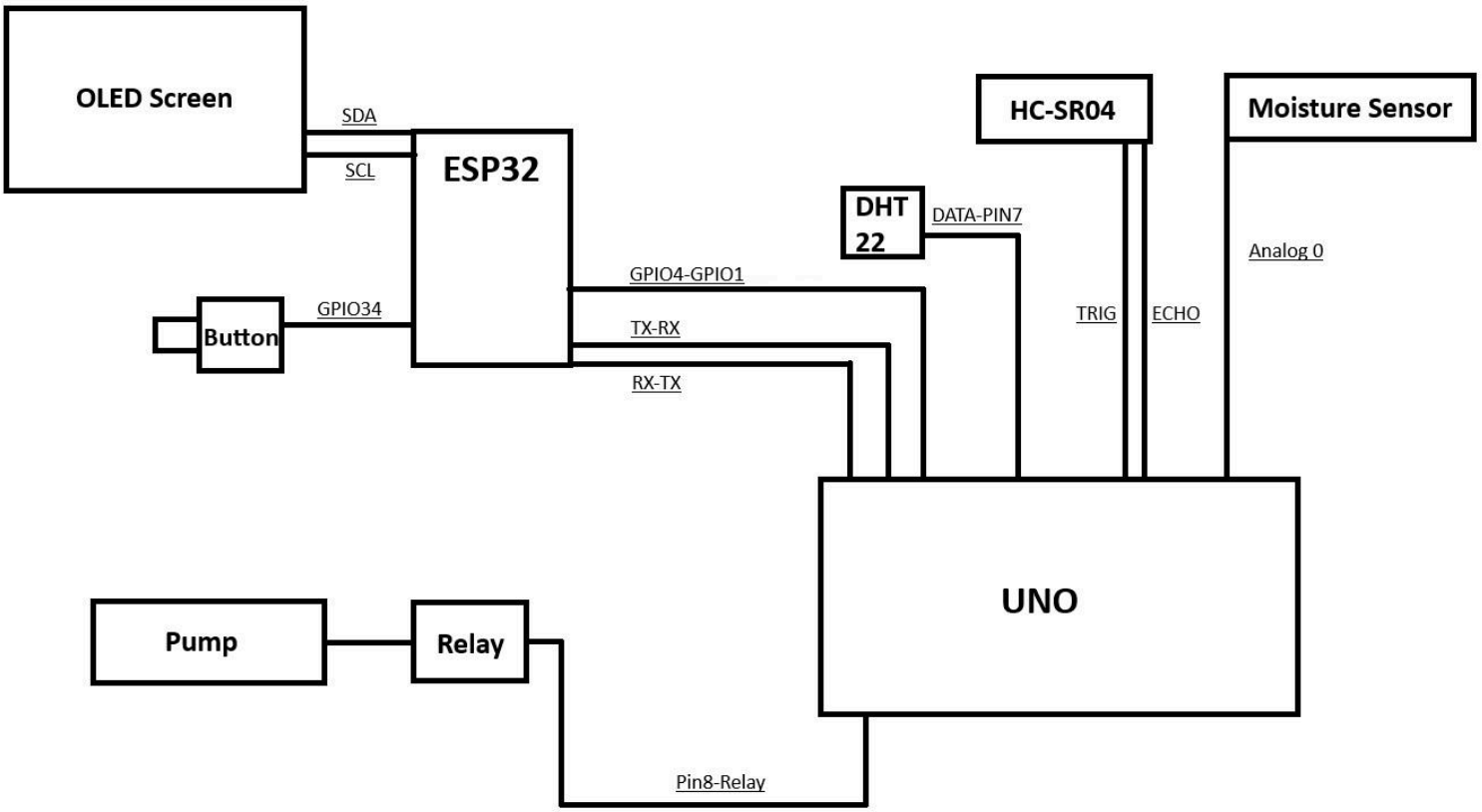
{"M":47,"T":23.6,"H":58.2,"D":4}

This information is then read by the ESP32, stored to values that determine the displayed values on the OLED and the moisture is then checked. If it is too low the ESP32 sends a command

{"cmd":"RELAY_ON"}

and when the UNO receives this command it turns the relay pin8 high for three seconds which connects a 5v pump to ground and waters the plant.

This image shows the basic control structure of the system.



3.2 Sensors

We used three sensors. The DHT22 for temperature and humidity of the room, the HC-SR04 time of flight sensor to measure the current water level and a capacitive moisture sensor to monitor the soil moisture. Each of these were connected to the UNO and powered by a shared 5V rail from the battery.

The DHT22 uses its own single line communication method. The UNO sends a start signal and receives back the information from the DHT. A 10kOhm pull-up resistor was used to stabilize the output. In the code, a DTH library is used to operate with the sensor.

HC-SR04 was connected with an Echo line and a Trig line. Trig line sends a start signal, the module sends out eight pulses and measures the return time and sends the information on Echo line. A library was used for the sensor.

The Moisture sensor measures the capacitance of the soil and sends back data when requested with a start signal over a single data line. For this sensor we wanted to be certain that the device wasn't over watering the plant by accident. We take a simple average of ten analog readings

```
long sum = 0;
for (int i = 0; i < 10; i++) {
  sum += analogRead(MOISTURE_PIN);
  delay(5);

int moistureValue = sum / 10;
int moisturePercent = map(moistureValue, 1023, 0, 0, 100);
```

The moisture levels were segmented in 1024 steps and mapped from 0-100%.

3.3 OLED Screen

The OLED screen was connected to ESP using SDA and SCL lines. An Adafruit library for the display was used for ease of coding. The screen was updated when the user pressed the button on the side of the device. A timer for two minutes was started and the screen would display the JSON string data. It would also display a warning message if the water in its reservoir was too low. After two minutes the screen was cleared to save power and avoid burn in.

4. Results

The system was implemented, and was mostly a success on the core functionalities. It was capable of collecting data, sending it to the ESP, saving it, displaying it and acting upon the data automatically. Some physical and code issues did occur however. After implementing the sleep mode on the UNO, we did not reliably get data from the DHT22. The other sensor did however work without issues. We believe that the issue was that we kept providing the sensor 5V directly even after the UNO was sleeping. The DHT22 therefore did not have a proper start up sequence and failed to communicate with the UNO. In the future we could solve this by providing the sensors power from a rail controlled by the UNO. By shutting down this rail, we would also see slight energy savings.

The other issue we faced was an issue with the 5V pump. It could not in reality pump water through the pipe, and instead halted quickly after a slight movement of the water. This issue was caused by a combination of using a boost converter, which lowered the amperage the batteries could deliver, and the 3,7V batteries having too low of a C rating to begin with. Our circuit could have feasibly provided enough current to operate the pump, but the start up current for a pump is much higher than this. A pump on start-up is equivalent to a motor with a sizable load attached to it on start-up due to the air and water it needs to move upwards. This could be fixed by more carefully considering the batteries used for the project.

5. Components used

- Arduino Uno

Microcontroller board that houses an ATmega328P, featuring digital and analog I/O pins, clock and programmable USB interface.

- ESP32

Microcontroller with integrated WiFi and bluetooth capabilities, dual core processing and multiple digital and analog peripherals.

- DHT22

A digital temperature and humidity sensor.

- HC-SR04

An ultrasonic distance sensor. Distance is measured by sending a sound wave and measuring the time of the reflected wave.

- Capacitive soil moisture sensor

A sensor that measures soil moisture by detecting changes in capacitance by varying levels of water in the soil.

- Relay module

Switching device that allows a low-power control signal to switch higher-current loads.

- 5V water pump

Small DC-powered pump to move liquid at low voltage.

- OLED display

A display module providing high-contrast and low power consumption that doesn’t need backlighting.

- Rechargeable battery

Portable energy storage that can be recharged and used to power the system

- Resistors and wiring

Passive electrical components to connect electronics

6. Programming Codes for Arduino and ESP

The code for the ESP and the UNO are stored in this github repository as two files.
https://github.com/olivervy-met/CyberPhysicalSystems-Project